

深信服，前端工程师岗，一面面经（40分钟）

深信服前端一面（40分钟）

1. 看过axios源码吗
2. 编码题
3. 判断是否为有效括号
4. 数组转化为树
5. webpack对哪些loader，插件比较了解
6. 自己有写过loader和plugin吗
7. webpack如何把没有用到的文件删除掉
8. 看过vue框架原理吗，说说你对vue的了解
9. 原生js发送ajax请求的方法，流程（XMLHttpRequest）
10. 前端安全防范措施，如何避免xss和csrf攻击
11. 跨域常用解决方案
12. 如何封装一个组件，要做什么准备（这里说的有点乱）
13. 箭头函数和普通函数的区别
14. 性能优化
15. 有哪些常用调试技巧？

参考答案：

3. 判断是否为有效括号

判断一个字符串是否为有效括号：

```
1  function isValid(s) {  
2      const stack = [];  
3      const map = {
```

```

4      '(': ')',
5      '{': '}',
6      '[': ']'
7  };
8
9  for (let i = 0; i < s.length; i++) {
10     if (map[s[i]]) {
11         stack.push(map[s[i]]);
12     } else {
13         if (stack.pop() !== s[i]) {
14             return false;
15         }
16     }
17 }
18
19 return stack.length === 0;
20 }

```

4. 数组转化为树

将数组转化为树结构：

```

1  function arrayToTree(items) {
2      const result = [];
3      const itemMap = {};
4
5      items.forEach(item => {
6          itemMap[item.id] = { ...item, children: [] };
7      });
8
9      items.forEach(item => {
10         const id = item.id;
11         const parentId = item.parentId;
12         const treeItem = itemMap[id];
13
14         if (parentId === null) {
15             result.push(treeItem);
16         } else {
17             itemMap[parentId].children.push(treeItem);
18         }
19     });
20
21     return result;
22 }

```

5. webpack对哪些loader，插件比较了解

常用的webpack loader和插件包括：

- Loaders:
 - `babel-loader`：用于转译ES6+代码。
 - `css-loader`：解析CSS文件。
 - `style-loader`：将CSS注入到DOM中。
 - `file-loader`：处理文件导入（如图片、字体）。
 - `url-loader`：类似于file-loader，但可以将较小的文件转换为base64。
- Plugins:
 - `HtmlWebpackPlugin`：生成HTML文件，并自动引入打包后的资源。
 - `CleanWebpackPlugin`：在每次构建前清理输出目录。
 - `MiniCssExtractPlugin`：将CSS提取为单独文件。
 - `TerserPlugin`：用于压缩JavaScript代码。
 - `DefinePlugin`：定义环境变量。

6. 自己有写过loader和plugin吗

是的，我写过简单的loader和plugin。写loader主要是处理特定类型的文件，例如自定义处理SVG文件。写plugin则是为webpack的构建流程添加自定义的功能，例如生成额外的报告文件。

7. webpack如何把没有用到的文件删除掉

webpack中，可以通过 `PurgeCSS` 等工具来删除未使用的CSS，或者通过 `webpack-bundle-analyzer` 来分析并优化打包体积。也可以通过配置 `optimization` 的 `sideEffects` 选项，以及使用 `TerserPlugin` 来移除未使用的代码。

8. 看过vue框架原理吗，说说你对vue的了解

是的，我了解Vue的框架原理。Vue是一个渐进式的JavaScript框架，用于构建用户界面。它的核心原理包括响应式数据绑定、虚拟DOM和组件化。响应式系统通过数据劫持实现，虚拟DOM则通过diff算法来高效更新DOM。Vue还提供了丰富的组件系统，支持组件的声明式渲染和复用。

9. 原生js发送ajax请求的方法，流程（XMLHttpRequest）

使用原生JavaScript发送AJAX请求的步骤：

```
1  const xhr = new XMLHttpRequest();
2  xhr.open('GET', 'url', true);
3  xhr.onreadystatechange = function() {
4    if (xhr.readyState === 4 && xhr.status === 200) {
5      console.log(xhr.responseText);
6    }
  }
```

```
7  };  
8  xhr.send();
```

流程包括：

1. 创建XMLHttpRequest对象。
2. 调用 `open` 方法初始化请求。
3. 设置回调函数处理响应。
4. 调用 `send` 方法发送请求。
10. 前端安全防范措施，如何避免XSS和CSRF攻击

- XSS防范：
 - 使用 `innerText` 或 `textContent` 代替 `innerHTML`。
 - 对用户输入进行严格的校验和过滤。
 - 使用内容安全策略（CSP）。
- CSRF防范：
 - 使用CSRF Token。
 - 使用SameSite Cookie属性。
 - 验证请求来源。

11. 跨域常用解决方案

- CORS（跨域资源共享）：通过设置服务器响应头 `Access-Control-Allow-Origin` 等。
- JSONP：利用 `script` 标签不受同源策略限制，支持GET请求。
- 服务器代理：通过服务器代理转发请求。
- `postMessage`：在不同域之间通过消息传递进行通信。

12. 如何封装一个组件，要做什么准备

封装一个组件需要以下准备：

1. 确定组件的功能和界面：明确组件的用途和设计。
2. 定义组件的API：包括属性、事件和方法。
3. 编写组件代码：使用Vue、React等框架创建组件。
4. 测试组件：编写单元测试，确保组件的功能正确。
5. 编写文档：提供使用指南和API说明。

13. 箭头函数和普通函数的区别

- `this` 绑定：箭头函数没有自己的 `this`，它会继承外层作用域的 `this`。普通函数的 `this` 由调用方式决定。
- 构造函数：箭头函数不能用作构造函数，不能使用 `new` 关键字。普通函数可以。
- `arguments` 对象：箭头函数没有 `arguments` 对象，普通函数有。

14. 性能优化

前端性能优化包括：

- 减少HTTP请求：合并文件，使用CDN。
- 压缩资源：使用Gzip压缩，压缩JavaScript、CSS和图片。
- 缓存：利用浏览器缓存、服务端缓存。
- 异步加载资源：使用 `async` 和 `defer` 加载脚本。
- 代码分割：按需加载，使用动态导入。

15. 有哪些常用调试技巧？

常用调试技巧包括：

- 使用浏览器开发者工具：查看DOM、控制台日志、网络请求、性能分析等。
- 断点调试：设置断点、条件断点和调试语句。
- 调试日志：使用 `console.log`、`console.warn`、`console.error` 记录调试信息。
- 调试源代码：通过Source Maps调试编译后的代码。
- 监视表达式：在调试过程中监视变量和表达式的值。
- 网络调试：查看网络请求和响应，检查接口是否正确。
- 性能分析：使用性能分析工具，找出性能瓶颈。